CS3213 Foundations of S. Engineering (FSE)

AY24/25 Sem 2. github.com/gerteck

- Software development processes (agile vs. plan-driven)
- Requirements engineering, Modeling, Software architectures
- Software testing, Static analysis, Debugging, fault localization.

1. Software Engineering

- **Software engineering**: *practicalities* of developing, delivering useful software. Programming integrated over time. Software sustainability to keep software evolvable over time. People as first-order success driver. Fred B. – no silver bullet.
- **Complexity**: *Essential complexity* - inherent, e.g. work w. stakeholders to determine product. *Accidental complexity* - solvable, e.g. buffer overflow exploitation protection. Hardest single part – deciding what to build.
- **Central Software Engineering Activities**: **Software Specification, Development, Validation, Evolution**: "Shifting Left" - more costly to address a defect late in the process. SE processes have been "shifting left".
- **SWE Personal Pinciples**: Humility, Respect/Trust. Thrive in ambiguity, value feedback, challenge status quo, user first. Improving, Passionate, Knowledge about People/Org, Sees Forest & Trees, Create shared context, safe haven. Working alone increases risks, design mistakes, irrelevance, and "bus factor."
- **Software Product Attributes**: *Elegant*: Simple, intuitive designs: easy to understand, maintain. *Anticipates Needs and future requirements*.

2. Software Engineering Processes (Process Model - sequence of activities)

- **Waterfall Model**: Simple, linear plan-driven software process model. Output of one phase feeds into next. Development starts only after req. eng and design. **Pros/Use Cases**: Embedded-hardware, Safety, Large system. Works if requirements clear, large project / distributed teams. **Cons**: accommodate change difficult.
- **Incremental Development**: Overlapping phases of specification, development, and validation. Built in increments, adding functionality. Can be plan-driven, agile, or mix. **Pros**: Lower cost of accommodating change, faster delivery. **Cons**: Harder to measure progress, may lead to system degradation (require refactoring).
- **Integration and Configuration / Reuse-based**: Reuse existing components/framework/library over from scratch. May need redefine reqs. based on identified components. **Pros**: Reduce amt. to develop, lower cost, risk. **Cons**: Limit customizatn, third-party dependency, integration complexity.

Agile Software Engineering (Agile Framework - not specific to software dev)

- **Agile**: Mindset and concrete management framework. Deliver value quickly, avoid waste. Use feedback loops, customer involvement. Adaptive to change (changing, unclear reqs). Lightweight process, empower people.
- **Origins**: 2001: Agile Manifesto by reps from frameworks like Scrum, XP, Feature-Driven Development (FDD). Influenced by *lean manufacturing*, – overlapping stages, built in instability, subtle control, self-organizing teams (autonomy, self-transcendence of teams, cross-fertilization, learning transfer cross teams).
- **Lean software development**: Limit waste: – aka partially done work, non-value adding processes/features, task switch, waiting for input, defects, handoff motion.
- **Manifesto Principles**: Individuals & interactions over processes. Working software over documents. deliver frequently. Customer collab over contract, welcome changing reqs. Working software as progress measure. sustainable development. Good design, simplicity enhances agility. Regular reflection.
- **Agile Adoption** : Larger orgs more likely to use hybrid. Bigger teams likely still use waterfall. Agile has better collaboration, alignment to business needs, better quality software. **BUT Problems**: business side slow to embrace. Too many mixed systems, siloed teams delays. Incorrect practice, managment buy-in, pain of lacking documentation.

Scrum ('Agile synonym'. Framework for project management - rugby term)

- **Core Principles**: Deliver value through increments in sprints.
- **Scrum Pillars**: Transparency, Inspection, Adaptation.
- **Scrum Team**: Self-managing, no hierarchies, $\leq$ 10members. *Scrum Master*: Facilitates Scrum process, removes impediments. *Product Owner*: Maximizes product value, manages Backlog. *Developers*: create Increments per Sprint.
- **Scrum Events**: *Sprint*: Timeboxed iteration ($\leq$ 1 month), deliver working Increment. *Sprint Planning*: Defines Sprint Goal, selects backlog items. *Daily Scrum*: 15-minute daily sync to track progress and adjust plan. *Sprint Review*: Team present results, adjust Product Backlog. *Sprint Retrospective*: Reflect on improvements for future Sprints.
- **Scrum Artifacts**: *Product Backlog*: Ordered list of work items aligned with Product Goal. *Sprint Backlog*: Selected items for Sprint w. actionable plan. *Increment*.
- **Timeboxing**: Sprint(S): $\leq$ 1 mnth. S. Planning: $\leq$ 8 hrs. Daily Scrum: 15 minutes. S. Review: $\leq$ 4 hrs. S. Retrospective: $\leq$ 3 hrs.
- **Kanban Board**: Used to visualize workflow. Help track progress, limit work in progress (WIP), improve flow (movement of potential value through a system).
- **Definition of Workflow (DoW)**: A shared understanding of how work items flow through the system. Includes *Units of value* (user stories), defined stages work items flow through, policies for how items move across state. WIP limits.

Extreme Programming (XP) - (Focus on Software Engineering Practices)

Highly influential agile methodology (late 1990s). "extreme" approach to iterative development and best practices. New ver may built several times/day, increments every 2 wks, All tests successful for accepted builds.

- **Pair Programming**: One programming, other review. (driver & navigator). Fewer bugs, better code understanding, learning, but lower efficiency, personal clash.
- **Collective Ownership**: Everyone responsible for all code, anyone can change any code. Pair prog facilitates this, as everyone better understands larger system.
- **Test-driven development**: Unit tests written before code, test all potential fail cases. helps development, facilitates other CI, refactoring etc.
- **Continuous Integration**: Dev team work on latest ver. Integrate code to main branch freq. Facilitate using unit tests. (GH Actions, Jenkins).
- **Simple Design, Refactoring**: "simple is best", refactoring as part of the lifecycle, keep it maintainable, easier to extend.
- Others: on-site customer for access system requirements, sustainable pace.

DevOps (Dev-scrum team and Operations Team)

- **DORA (DevOps Research and Assessment)**: Framework for measuring and improving DevOps performance. Key metrics: deployment frequency, change lead time, change failure rates, mean time to recovery.
- **Silos**: dev and ops teams separated, conflicting goals. Developers deliver features quickly, while operations want stability and uptime. DevOps fosters collaboration.
- **DevOps Overview**: Emphasize automation, collaboration, continuous delivery. Routine, predictable deployments. Agile is on iterative dev, DevOps on automating deployment pipeline. CI, delivery, feedback loops. "Infinite loop" of dev-ops.
- **Principles**: *Flow*: Reduce batch sizes & handoffs (work passing across depts/teams) to streamline delivery. *Feedback*: Fast feedback loops, resolve issues early. *Continuous Learning*: Mechanisms to share knowledge. Safety culture. *blameless postmortems* for failures.
- **Value Stream Mapping**: Visualize, analyze the flow of work, identify inefficiencies, improve delivery. (E.g. develop unit test $\rightarrow$ code solution $\rightarrow$ PR reviews $\rightarrow$ QA $\rightarrow$ regression, performance test $\rightarrow$ deploy $\rightarrow$ smoke test $\rightarrow$ UAT test)
- **CI/CD**: Automate integration, testing, and deployment of code changes, enabling frequent and reliable releases (push to production). *Infrastructure as Code*: Treats infra configuration like code, use version control, automation to manage servers and environments.

3. Software Requirements Engineering

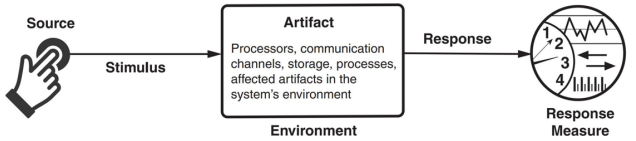
- **Requirements Engineering (RE)**: Discovering, analyzing, documenting, validating services and constraints. Focus on problem space, not solution space.
- **Wicked Problem**: Requirements are ambiguous, conflicting, evolving. No single correct solution. Hard to define, solve, or validate them definitively.

Requirement Engineering (RE), Why & Challenges

- **High Cost of Errors**: Errors during requirements phase costly.
- **Importance of RE**: Facilitates communication between stakeholders. Lay foundation for structure, behavior of software system. Basis for testing, accepting final product. Defines quality of system, risks, and operational overhead.
- **RE Challenges**: Difficult to determine if all relevant requirements are collected. Resource-intensive, often one-shot effort. Cannot be generalized.
  - *User-Related*: conflicted views, incomplete understandg of needs, limits.
  - *Analyst-Related*: Poor problem domain knowledge, omit "obvious" info.
  - *Communication*: Requirements evolve w. time, Ill-defined system boundary. Vague, untestable requirements ("user-friendly," "robust").
- *NaPiRE* (RE pains): Incomplete/hidden req, comm. flaws, time-boxing (not enough time), moving targets, inconsistent/underspecified req...

Functional vs. Non-Functional Requirements (NFRs - 'Quality Attributes')

- **Functional Requirements**: Describe what system should do. Stakeholders typically focus on functionality. Functionality does not determine system architecture.
- **Non-functional Requirements**: Define system qualities, not specific services (e.g., performance, security, usability). Often more critical than FRs, may involve implicit stakeholder expectations. Can conflict (e.g., security vs. usability). Influence architectural tradeoffs (e.g., *quality attributes*).
- Non-functional requirements should be quantitative to be testable. E.g. ("throughput sufficiently high." vs. "throughput to exceed 1,000 query/sec")
- **(Framework for NFRS) Quality Attribute Scenarios**: framework to clarify, document NFRs. Up to 6 elements: (*Stimulus*: An event arriving at the system. *Source*: Origin of stimulus (e.g., human, system), *Artifact*: part of system affected by stimulus, *Response*: system's reaction. *Response Measure*: Quantifiable metrics for response. *Environment*: scenario context).



Example Non-Functional Requirements and Metrics

- **Availability**: System readiness to perform tasks, reliability and recoverability. *Metrics*: Availability %, time to detect/repair faults, etc. (QA Scenario: E.g. 1. Source: server, 2. Stimulus: crashes, 3. Artifact: the server software, 4. Environment: any operational state, 5. Response: detects the abnormal state and re-routes traffic to another server, 6. Response measure: within 5 seconds of the server becoming unresponsive).
- **Performance**: System responsiveness. *Metrics*: Latency, throughput etc.
- **Modifiability**: Cost and effort. *Metrics*: Elapsed time, new defects introduced.
- **Deployability**: The ease of deploying and rolling back system changes. *Metrics*: Cost, percentage of failed deployments, cycle time, etc.
- **Security**: Protecting data and systems from unauthorized access. (CIA Triad) *Metrics*: Accuracy of attack detection, size of trusted execution

Requirement Engineering Process Models

Agile Approaches: Agile avoids detailed requirements documents due to rapid changes. Relies on user stories and frequent stakeholder feedback. Iterative feedback, collaboration. Short supporting documents may still define business and dependability requirements.
Plan-Driven Approaches: Follow a structured process model. RE phase is completed before implementation. Produces a requirements document, often part of the development contract. Focus on detailed specification and validation

SRS: Software Requirements Specification: Central document of what to implement. Detailed specification of system/user requirements. Output of RE.

Stakeholders: impacted / influencing requirements. Start with "baseline" stakeholders (e.g., users, decision-makers). Explore network to identify more.

RE Key Activities - 3Cs of User Stories

Elicitation and analysis → Conversation (w. client).
Specification → Card (description).
Validation → Confirmation (acceptance tests to determine completeness).

1. **Elicitation and Analysis**: Get Requirements working w. stakeholders.
   Elicitation techniques: Interview, Existing Docs Analysis, (user-story) workshops – multiple stakeholders & resource intensive, FGs (users), prototype, observe – discover implicit req. reflecting actual ways. Personas.

   - **RE Elicitation and Analysis Model**:
     Requirements Discovery (Interact w. stakeholders to uncover needs) → *Classification and Organization* (Group req. to coherent clusters) → *Prioritization and Negotiation* (Resolve conflicts, prioritize req) → *Documentation* (Record requirements for next iteration) → (repeat)

2. **Specification**: Convert Requirements to standard form (user-centric view).
   User Stories: represent customer requirements (<As 'user-type', want 'goal', to 'reason'.>) **Three Cs** - card, convo, confirm. **Epics** – too large user stories to be split later. Include notes, NFRs (constraints).
   Use Cases: interaction btwn actor & system (e.g. check in flight). UML → use case diagram. May encompass multiple scenarios (single instance of usage e.g normal flow, crash, others etc). Natural Language – use standard format, consistency, associate rationales, avoid jargon.

   - **User vs. System Requirements**: **UR**: High-level, no software jargon descriptions of external behaviour of system. **SR**: Complete, detailed description of whole system, external behavior and operational constraints.

3. **Validation**: Ensure requirements actually define desired system.
   (*Agile*: freq comm. w stakeholder, *Plan*: check unclarities in req. document).
   Techniques: *Informal Reviews*: Peer review, colleague reviews requirements. Informal feedback collection (e.g., deskchecks, walkthroughs). *Formal Reviews*: Systematic analysis by a team of reviewers, roles: Moderator, reader, inspectors. Techniques: Defect checklists, requirement-by-requirement reviews.
   Requirement document checks: *Validity*: reflect real user current needs. *Consistency*: Avoid contradictory req. *Completeness*. *Realism*: feasible, within budget, tech constraints. *Verifiability*: testable.
   Feasibility Study: short study to assess: alignment with organizational objectives, budget and schedule, Integration with existing systems.
   Prototyping: Quick version of system to validate requirements and design decisions. Reduce post-delivery changes by allowing early experimentation.

   - **Requirements Change Management (During Validation)**:
     Established Plan for to manage evolving requirements.
     Unique identification of requirements (to cross-reference).
     Established change management process to assess impact and cost.
     Traceability policies to link requirements to design.
     Tool support for managing requirements

Specification and Validation

- **Use Cases vs User Stories** (Use Cases – Plan-Driven): Specify functional requirements and serve as test bases. (User Stories – Agile): Act as placeholders for implementation, elaborated into acceptance tests.
- **Requirements as a Basis for Testing**: Requirements should be testable and used to derive test cases. User stories, cases as foundations for acceptance tests.
- **V-Model (Plan-Driven Approaches)**: In plan-driven approaches, requirements validation also has test planning, and aligns with testing phases in the V-Model. (I.e. User Requirement + acceptance test planning ↔ acceptance test, Functional Req ↔ System Test, Architecture ↔ integration test, Design ↔ Unit Test.) Horizontal axis is time. Represented as V - shape to match.
- **Spiral Model**: iterative, plan-driven RE process model. Expands requirements incrementally, focusing on high-level business needs early and detailed system requirements later. Outputs SRS. Can incorporate agile in later iterations. (From inside spiral outwards. Repeated cycle of Elicitation, Specification and Validation)

4. System Modeling and Software Architecture

Systems Modeling

Develop abstract models of a system. Means to an end (developing software).

- Emphasized in plan-driven, in agile used in lightweight manner ("just enough").
- Formal models (mathematical) can be developed for auto verification.
- Models created during requirements engineering, system design, and post-implementation documentation.
- **Abstraction vs Representation**: Abstraction simplifies systems, omits details. Representation maintains all system information.
- **Different perspectives**: External, Interaction, Structural, Behavioral.

Modeling Languages (UML, C4, others: BPMN, SysML, etc.)

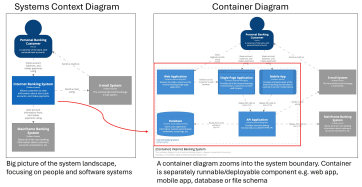
UML (Unified Modeling Language) Key Diagrams:

- Class Diagrams: Show object classes and associations.
- Sequence Diagrams: Show interactions btw. actors & components.
- Activity Diagrams: Show process or data flow activities.
- State Diagrams: Show system reactions to events.
- Use Case Diagrams: Show interactions btwn system & environm.

C4 Model: Agile Architectural Modeling. For software architectures. Do "just enough" design / upfront thinking for starting point/ general direction.

- Hierarchical architectural modeling (4Cs): System Context, Containers, Components, Code. Notation Independent.

- *System Context diagram* → starting point, how software fits into world.
- *Container diagram* → software system, high level building blocks.
- *Component diagram* → individual container, decomposes containers to identify major structural building blocks (components). Not recommend unless adds value.
- *Code diagram* (e.g. UML class) → indivdual component implementation. Not recommended for C4 / Agile context use.



Software Architecture (overall structure of a system, part of software design)

- **Influences quality attributes, modifiability, and communication** among stakeholders. Mitigate highest priority risks. Difficult to change later.
- **Architectural Drivers: Architecturally Significant Requirements (ASRs)**: Requirements that significantly impact architecture. Non-functional req. (e.g. availability, performance) often ASRs. Functionality must have major influence on architecture. (E.g. system should provide five nines (99.999 percent) availability).

Often, system architecture modeled informally w. simple block diagrams.

Architectural Reasons and Consequences

- **Quality Attributes**: Architecture significantly influences whether system meets quality attributes (non-functional requirements). (E.g. *High Performance* – require managing time-based behavior and shared resource access, *Safety and Security...*)
- **Modifiability and Change**: Modifiability is a critical quality attribute. Requires assigning responsibilities to components to limit the impact of changes. *Categories of changes*: Local – affect only single component, Non-local – Affect multiple components (e.g. UI). Architectural Change – Affects entire system (e.g., single-thread to multi-thread). Good architecture ensures most changes local or non-local.
- **Communication Among Stakeholders**: Architecture as common abstraction for mutual understanding, negotiation, and consensus.
- **Additional**: Enables reasoning of cost, schedule, basis for reusable models in product lines, focuses on component assembly, channels developer creativity by restricting design alternatives, foundation for training new team members, enable early prediction of system qualities, defines constraints for subsequent implementation, influences org structure (Conway's Law), supports incremental dev.
- **Conway's Law**: Organizations design systems that mirror their communication structures – Architecture often forms basis for work-breakdown structures.

Attribute Driven Design (ADD): Architecture Patterns and Tatics

Architecture is a tradeoff of attributes.
Architectural tactic: design decision that influences a quality attribute.
Architectural Patterns: Established architectural solution, may comprise multiple architectural tactics. Often found useful over time, documented and disseminated.

- **Attribute-Driven Design (ADD)**: Systematic approach to architecture design. Each design iteration, address an ASR.
  Steps: Identify ASRs → set iteration goals (satisfy which set of ASRs) → choose target structure → select designs (choose tatics, patterns, most difficult) → instantiate patterns in context → record decisions → validate, repeat.

- **Antipattern – Big Ball of Mud**: Common in practice. Lack of architectural design, Business pressure, Erosion of architecture over time. Poor maintainability, extensibility (Quality Attributes)

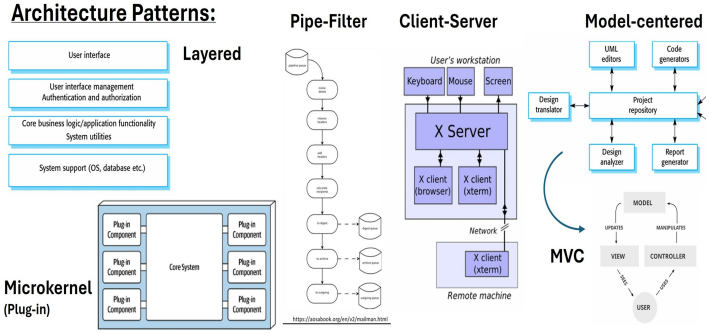
Common Architectural Patterns

- **Layered Architecture**: Structures systems into layers with defined interfaces. Layer only uses layer directly beneath, through public interface. (constraints may be violated e.g. for performance, layer bridging, upper layer bypass lower layers).
  Pros: portability (to diff env/OS), modifiability (lower layers changeable under hood, less interfaces to understand), reuse
  Cons: Layering possible performance penalty. If not properly designed, can impede (missing nec. low level abstraction needed at higher level).
- **Pipe-and-Filter**: Processes data through independent filters connected by pipes. *Filters*: processing components, take input, produce output, no assumption on other filters, stateless. *Pipes*: connect multiple filters with each other. (E.g. MailMan - add header, digest, archive, outgoing).
  Pros: modifiability (independent filters) and reconfigurability, evolution (add/combine filers differently)
  Cons: format data transfer must be set. performance as every filter must parse input convert to output format.

- **Model-Centered**: Independent components interact with a central model than each other. Repository style. (E.g. IDEs).
  Pros: components can be independent. highly modifiable. concurrency. State/data manged consistently (in one place).
  Cons: model/repo is single point of failure. change to model = many changes. Distributing repo can be difficult.

  MVC design pattern: "instantiation" of model-centered, goal of MVC to separate model (business logic) from UI (controller/view).
- **Microkernel (Plug-in)**: Base system, plug-in components (e.g. web browser).
  Pros: modifiability, extensibility, testability. Plugins can be developed, tested, evolved by diff. teams independently.
  Cons: can introduce security /privacy vulnerabilities

- **Client-Server**: central server, provide service to multiple distributed clients. Client request service, server cannot initiate. (E.g. web server) *Discovery*: client use discovery service, server respond w. agreed protocol. *Interaction*: client sends request, server process, respond.
  Pros: Low coupling btwn server clients, no coupling among clients. Easily scale no. clients and server. Both can evolve indptly.
- **Microservices**: Independently deployable services communicating via APIs. Typically stateless, small teams, small code bases. Service dependencies acyclic. Popularized by large companies. Goal - decoupling, microservices expected to be independent w own domain/workflow. typically smaller than services in SOA.
  Pros: quick time to market, indiv. deployability, test, scalability.
  Cons: network comm. overhead, complex transactions, challenge in design /maintain sea of microservices



Monoliths vs. Service-Oriented Architecture (SOA): Monolith typically single deployable unit (diff. way of partition e.g. layered archi.). SOA for distributed-architecture styles, services separately deployed, only interact over interfaces, network. Related to microservices.

Architectural Tactics

Tatics are design decisions that influence quality attributes. E.g.

- *Availability Tactics*: Detect faults (monitoring, heartbeat), recover (redundant spare, rollback), prevent faults (ACID transactions, remove from service).
- *Performance Tactics*: Control Resource Demand: (Manage work requests, limit event response, bound execution times), Manage Resources: (maintain multiple copies of data, schedule resources, bound queue sizes, introduce concurrency).

"20 minutes rule": Continuous learning essential for staying updated.

5. Software Testing

Purpose of Testing: *Defect Testing*: Find bugs in software. (Tests show presence, but never absence.) *Validation Testing*: Ensure system meets user requirements.

- **Verification vs. Validation**: *Verify*: Build product right, conforms to specification. *Validate*: Build right product, software meets user needs.
- **Test Case**: A test case consists of: *Test Input*: Arguments, system state, or actions, *Test Oracle*: Mechanism, determine test pass or fail.
- **Test Oracle**: Can check functional properties (e.g., correctness) or non-functional properties (e.g., performance, security).
- **Manual vs. Automated Testing**: *Automated*: Use tools to auto generate inputs and apply oracles (e.g., JUnit). *Manual*: Tester manually inputs test cases and acts as oracle. ('Traditional' - auto as in test automation).
- **Pesticide Paradox (Diminishing Return)**: 'Every method used to prevent/find bugs leaves residue of subtler bugs which those methods ineffectual'. Diff testing methods have pros, cons. Use many, similar to pests build immunity.

Testing Levels, Flaky Tests

1. **Unit Tests**: Test indiv. components (e.g., single method/class).
   Pros: Fast execution, easy control, write (no additional setup (e.g., no DB).
   Cons: Lack realism, don't represent real system execution. May miss integration bugs, often require mocks to simulate external dependencies.
2. **Integration Tests**: Test interactions btw. multiple components. Focus on integrating target cmpnt. w. external cmpnt. (e.g., ext DB, web service). - (e.g. SQLancer test query plan converted to expected internal representation.)
3. **System Tests**: Test entire system as whole. *Pro*: realistic test env. *Cons*: slower execution, harder to write (complex setup), prone to flakiness (non-deterministic results). (e.g: Ensuring SQLancer compatibility with database system.)

- **Test Flakiness**: Pass/fail non-deterministically. Low result confidence.
  Causes: Concurrency issue (e.g., race conditions), async waits (e.g., not properly waiting for server responses), test order dependencies (e.g., improper cleanup), test timeouts, resource leaks. (e.g, test fail from `ConcurrentModificationException` in multi-threaded code.)
- **Test Pyramid**: strategy for balancing test granularity. Many small, fast unit tests as base. Fewer integration, system tests. *Antipatterns*: Too many high-level tests, slow execution, maintenance challenges.

- **E.g. Test Sizes at Google**: Tests classified by size to for efficiency, determinism.
  Small: Single process/thread, no I/O, fast, deterministic.
  Medium: Multiple processes, allow localhost network calls.
  Large: No restrictions, slower, less deterministic.

Testing and Processes

- **Plan Driven Development**:
  - **V-Model**: extension of waterfall model. Tests planned alongside RE.
  - **User Acceptance Testing**: validate system meets user req, w customer participation. In *plan driven*, explicit phase after system implementation. In *agile*, UAT is interleaved (e.g., acceptance criteria in user stories).
- **Test-Driven Development (TDD)**: Emphasized by XP. Use as design process, focus 'happy path'. *Pros*: Focus on req, provide quick feedback. Encourages testable code, allows incremental exploration.
  Steps: Write test → ensure fails → simplest code to pass → refactor.
  When to Use: Useful for complex prob, unclear soln (incremental experimentation). Less useful if solution already clear.
  Schools of thought: (Classicist/Detroit, Inside-out start e.g. business rules → UI). (London/Mockist Outside-In: Start outside, use mocks).
- **Black-box vs. White-box Testing**:
  Black-box 'specification testing': No internal info, based on requirements.
  White-box 'structural testing': Internal info, focus code structure, logic.

Black-box Testing (Specification Based Testing)

- **Specification-based Testing**: Derive from requirements (e.g., user stories, use cases, docs). Functional & non-functional requirements.
  Testing Workflow: Understand requirements (input, output), explore program (functionality), identify partitions (eqv classes of input/output), analyze boundaries, devise and automate test cases, augment test suite with common bugs.
- **Equivalence Partitioning**: Divides inputs into equivalent classes. Output partition may help spot input partition. Combinations of partitions of individual inputs.
  - E.g. Partitions for STR: null, empty, single-char string, multi-char string.
- **Boundary Value Analysis**: Inputs at boundaries more likely to trigger bugs (e.g. off by one errors). Program behavior changes when going from one boundary to the other. Tests edge cases (e.g., min/max).
  - E.g. For given partitions, choose boundaries. $(+1, -1, =)$

White-box Testing (Structural + Mutation Testing)

- **Structural Testing**: use source code to guide testing. Catch requirement independent implementation aspect (e.g. cache, DSA). Focus on coverage criteria.
- **Coverage Criteria**:
  - *Line Coverage* (percentage executed).
  - *Branch Coverage* (branch percentage executed).
  - *Branch + Condition Coverage*: Considers every condition of each branch statement. E.g. `if (a||b)`. Condition coverage does not imply branch coverage.
  - *Path Coverage*: Strongest criterion. Often impossible (e.g. loop). In simple function with 10 conditions, number of paths = 1024
  - *Modified Condition/Decision Coverage (MC/DC) Coverage*: Test important combinations of conditions, with less cost.
  - Overall, Path Coverage > MC/DC > Branch+Condition ... (subsumes/includes)

MC/DC Coverage

$$\frac{\text{Number of conditions independently tested to affect decision outcome}}{\text{total number of conditions in decisions}} \times 100\%$$

- For each decision (Boolean expression), identify all conditions. Count conditions:
  1. Tested with both true and false values.
  2. Proven to independently affect the decision's result.
- (For a decision with n conditions, $n+1$ test cases lower bound to demonstrate independence for each condition.)

MC/DC: Modified Condition/Decision Coverage

Test case	A AND B	result	applying MCDC rules
1	1 1	1	A and B are decisive for the result "1"
2	1 0	0	B is decisive for the result "0"
3	0 1	0	A is decisive for the result "0"
4	0 0	0	This test case can be skipped based on the MCDC-rules, as both A and B have been decisive for both a true and a false result of the complex condition

- **Mutation Testing**: checks how good test cases are by intentionally adding small bugs (mutants) to code and seeing if tests can catch/kill them.
  Steps: Select statement in code, mutate code, run test suite, record outcome. Undo change, repeat. Calculate mutation score: $\frac{\#\text{killed mutants}}{\#\text{mutants}} \times 100$.
  Pros: Identify undertested code.
  Cons: Comptn expensive; equivalent mutants may not change behavior.

6. Debugging and Fault Localization

Debugging

- **Shifting Left**: Debug during software development (e.g., due to failing tests or static analysis reports), but also after deployment (user reports).
- **Typical strategy to debug include**:
  - **Printf debugging**: Simple, intuitive, language tool agnostic. But can quickly be confusing, forget to remove print, may require recompilation.
  - **Logging**: More systematic alternative to print statements. Multiple debugging levels supported (E.g. INFO, WARN, ERROR, FATAL).
  - **Debugger**: Language-specific. Set breakpoints, stepping in/over methods, step-wise execution, program state (variables, stack trace.). More systematic, more info. However, still require strategy with debuggers.
- **Challenges**: Program states very large, challenging to search all manually. Not clear if (intermediate) states correct or not. Execution could have millions of steps. Hence, need good debug strategy to focus search.

Fault vs. Defect vs. Failure:

- **Fault**: Location of flaw in code, causes incorrect behavior/unwanted state.
- **Defect (Bug)**: Manifestation of fault in software. (Error in program code)
- **Failure**: Deviation of behavior from expected output. (Externally visible)

To debug a failure, trace back to defect that caused the first fault.

Scientific Method to Debugging (Systematic Approach)

Systematic process of identifying faults. Basically, observe failure, generate hypotheses on cause, test (e.g., add debug , assert statements). Confirm results.

1. Formulate a question.
2. Come up w. hypothesis based on knowledge to explain.
3. Determine logical consequences of hypothesis, formulate prediction that supports or refutes it.
4. Test prediction in experiment. Confidence in hypothesis increase or decrease.
5. Repeat until no discrepancies between hypothesis, predictions, observations.

- **Scientific process Theory**: a hypothesis that is consistent with earlier observations, predict future obsv (program behavior). → Diagnosis.
- **Diagnosis** (Debugging): explains both causality (why), and incorrectness.
- Systematic Debugging: helps to systematically log and create a table etc.

Rubberducking: Explaining code line-by-line to an inanimate object (or another person) to identify logical errors or misunderstandings.

Program Slicing

Identifying parts of program (slice) that influence/ are influenced by variable/ statement. Auto determine which earlier variables influenced current erroneous state.

- **Slicing**: extract subset of program potentially affecting variables at specific program location.
- **Advantages**: Narrows scope of debugging, rule out locations that cannot have effect on failure. Bring together possible origins that may be scattered across code. AKA helps consolidate scattered dependencies into a single, manageable view.
- **Types of Dependencies**: *Data Dependency*: Variable's value depends on prior variable. (e.g. fn params). *Control Dependency*: Execution of statement depends on condition. Can reason using program dependence graph.
- **Static vs. Dynamic Slicing**: *Static Slicing*: Considers all possible executions of the program. (e.g. include whole method, both if and else branch). *Dynamic Slicing*: Focuses on a specific execution path based on concrete inputs. (e.g. only consider if branch, as else branch not executed).
- **Forward vs. Backward Slicing**: *Backward Slicing*: Identifies what influenced a value. *Forward Slicing*: Which statements influenced by particular value.
- Slicing applications: As mental model, some support in IDEs, papers published, program understanding, maintenance, security analysis.

Debugging Strategy - Tracking Origins

- Start with faulty state f (failure), trace origins to earlier states possibly causing f.
- Identify if origins faulty or correct. (By inspecting individual state variables)
- Continue tracing till faulty state with only correct origins found, indicating defect.

Statistical Fault Localization

- **Observation**: Program failures correlate with events that precede them.
- **Tarantula Algorithm**: Assigns suspiciousness scores to program elements based on involvement in failing and passing test cases. Hue $= [0, 1]$
  - For each line of code:
  - Formula: [Suspiciousness(line) = $\frac{\frac{failed(line)}{totalfailed}}{\frac{failed(line)}{totalfailed} + \frac{passed(line)}{totalpassed}}$]
  - Visualize using color coding (red: suspicious, green: safe). Closer to 1 = sus.
  - failed(line) = number of failing tests that execute this line.
  - passed(line) = number of passing tests that execute this line.
- **Limitations**: Provides no direct info about root cause beyond suspicious lines. Effectiveness depends on complexity of bug. Limited IDE tool support.

Test-Case Reduction

Reduce large failure-inducing inputs to minimal reproducible test cases.

- **Small Scope Hypothesis**: "A high proportion of errors can be found by testing a program for all test inputs within some small scope". Most bugs triggerable w. small, simple inputs. Can reduce the bug-inducing input.
- **Test-case reduction tools**: reduce input element, apply (user-supplied) test oracle, check whether the bug is still being triggered If so, continue applying another reduction. If not, undoes change, attempts another reduction.
- **Reducer Characteristics**: Advantages: Simplifies debugging (easier to debug, analyze reason), aids in identifying duplicate bugs, and improves communication of bug. Challenges: Interestingness tests can be complex; inefficient for highly structured languages without hierarchical reduction techniques.

Delta Debugging

- **Like Binary Search w Variations**: Binary search alone might miss interactions (diff pieces tgt, or full input).
- **Goal**: Find 1-minimal input such that `test(c') = FAIL`, and for all proper subsets, `test(c'') ≠ FAIL (PASS or UNRESOLVED)`
- **1-Minimality**: no subset of reduced test causes test to fail. Finding global min has exponential complexity $(2^{|c\chi|})$, so Delta Debugging gives local minimum.
- Systematically **remove elements** from input while ensuring bug still triggered. Behaves like binary search but explores combo of smaller blocks. Iteratively increases granularity if neither partitions nor complements trigger the bug.
- Start reducing, consider granularity of n=2 (two halves). Consider partitions of input, and then complements (Whole input minus the specific partition). If any of partitions cause test to fail, continue reducing partition. Otherwise, reduce complement. If both do not work, double granularity (create more partitions).

Minimizing Delta Debugging Algorithm

Let *test* and $c_\times$ be given such that $test(\emptyset) = ✔ \wedge test(c_\times) = ✘$ hold.
The goal is to find $c'_\times = ddmin(c_\times)$ such that $c'_\times \subseteq c_\times$, $test(c'_\times) = ✘$, and $c'_\times$ is 1-minimal.
The *minimizing Delta Debugging algorithm* $ddmin(c)$ is

$$ddmin(c_\times) = ddmin_2(c_\times, 2) \quad \text{where}$$

$$ddmin_2(c'_\times, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \ldots, n\} \cdot test(\Delta_i) = ✘ \text{ (“reduce to subset”)} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \ldots, n\} \cdot test(\nabla_i) = ✘ \text{ (“reduce to complement”)} \\ ddmin_2(c'_\times, \min(|c'_\times|, 2n)) & \text{else if } n < |c'_\times| \text{ (“increase granularity”)} \\ c'_\times & \text{otherwise (“done”).} \end{cases}$$

where $\nabla_i = c'_\times - \Delta_i$, $c'_\times = \Delta_1 \cup \Delta_2 \cup \cdots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_\times|/n$ holds.
The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_\times) = ✘ \wedge n \leq |c'_\times|$.

Delta Debugging PseudoCode

```
function ddmin(c):
    n = 2
    while len(c) ≥ 2:
        split c into n subsets: Δ1, Δ2, ..., Δn
        for each Δi:
            if test(c - Δi) == FAIL:
                return ddmin(c - Δi)
        for each Δi:
            if test(Δi) == FAIL:
                return ddmin(Δi)
        if n == len(c):
            break
        n = min(len(c), n * 2)
    return c
```

Isolating Failure-Inducing Changes

- **Regression Bugs**: Features work in previous versions, fail in newer ones.
- **Git Bisect**: Automates the process of identifying the commit that introduced a regression bug using binary search.
  - Steps: Mark a known good version (`git bisect good`) and a bad version (`git bisect bad`). Test intermediate versions until the first bad commit is identified. Can be automated using scripts that build and test the program for each version.
- **Delta Debugging on Changes**: Applies delta debugging to individual patches within a commit. Granularities include hunks (consecutive differing lines) or entire changesets.

7. Software Development Practices

Development Practices: Asking 'How'. Credits: Guest Lecturer Kareem Shehata.

Source Control Management (SCM)

- **Questions to ask**: Who is allowed to view, change code? How are changes managed? dependencies managed? How does source control integrate with tools?
- **SCM Tool Considerations**: (source control ≠ git!)
  - **Centralized vs Decentralized**: Centralized systems have single repository, decentralized allow multiple repositories w. synchronization.
  - **Scalability**: Handle large repositories and teams, **Development Tool Support**: Integration with IDEs like VS, XCode, **Expertise/opinions in team**: Choose tools that align with whole team's expertise, preferences. **Dependency Management**: Tools (Maven, Gradle) help manage deps.
- **Good SCM Practices**:
  - **Single Source of Truth**: Maintain single authoritative repository. **Consistency**: Consistent naming conventions (branches, tags, commits). **Small, Focused Changes**: Commit small, independent changes that are easier to review and revert if necessary. **Clarity**: Clear commit messages explaining what was changed and why.
  - **Use Branches and Tags Appropriately**: Use branches for feature development and tags for releases. **Avoid Long-Lived Branches**: Merge frequently, reduce integration challenges. **Use Toolset for Managing Dependencies**: Leverage dependency management tools to handle external libraries.

Coding Style

Consistent coding style improves readability, maintainability, collaboration. Also, probably need to revisit, read own code in the future.

- **Why Care About Style**: Unnecessary changes in SCM due to formatting diff. Inconsistencies make code harder to read, understand. Style differences can lead to confusion and inefficiency.

- **Key Elements of a Style Guide**: **Consistency over Optimality**: Consistency more important than overly optimized code. **Focus on Clarity**, be clear and unsurprising. **Practicality over Clever solutions.** Include guidelines on how tasks should be approached, not just formatting rules. **Autoformatters**: Use tools like Prettier, clang-format, or Black to automate formatting.

Versioning and Releases

- **Releases**: Depends on deploy method. Web services w. rolling releases, app stores, binaries, embed systems will be diff. in deployment, releases.
- **Software Versioning**: Process of assigning unique identifiers to bundles of software. Identifiers help track, manage diff releases of software over time. Goals:
  - Allow users to know which version of the software using, enable developers to identify version customers are using.
  - Managing dependencies between software components.

Semantic Versioning (SemVer)

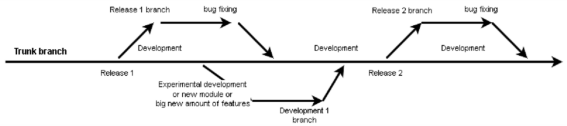
Semantic Versioning is a widely adopted versioning scheme, particularly for software with public APIs. It uses a three-part version number: **MAJOR.MINOR.PATCH**.

- **MAJOR version**: Increment: incompatible API changes are made.
- **MINOR version**: Increment: backward-compatible functionality is added.
- **PATCH version**: Increment: backward-compatible bug fixes.
- **Hyrum's Law**: With a sufficient number of users of an API, it does not matter what you promise in the contract: all observable behaviours of your system will be depended on by somebody.

System ensures clarity and consistency in communicating changes to users and devs. However, these labels are slapped on by a human - no perfect guarantee.

Versioning Schemes

- **Commits as Identifiers**: Source control systems like Git assign unique hashes to commits (e.g., `2d54f09b`). **Pros**: Automatic, no additional effort required; useful for internal tracking. **Cons**: Not all commits correspond to release versions; difficult to remember and interpret.
- **Assigned Identifiers**: Includes code names (e.g., Android's "Lollipop," "Marshmallow") or version increments (e.g., v1, v2, v3). Can be combined with tags indicating release stages (e.g., alpha, beta, rc release candidate). **Pros**: Easier to remember; code names aid branding and communication. **Cons**: Requires manual assignment and management.
- **E.g. Versioning in Practice**: **Android**: Uses both numeric versions and dessert-themed code names (e.g., Android 9: Pie). Code names follow alphabet order, aid in identifying newer versions. **macOS**: Combines numeric versions with thematic names (e.g., macOS Big Sur, macOS Monterey). **Ubuntu**: Uses calendar-based versioning (YY.MM) and distinguishes Long-Term Support (LTS) releases.
- **Choosing Versioning Scheme**: Depends on organizational needs, deployment strategies, release frequency, and user expectations. For example: web services with rolling releases may prefer frequent, small increments. Embedded systems/ safety-critical software may strict versioning/ long-term support. (Needs of marketing / users?)
- **Best branching strategy**: Anything that gives a single source of truth, consistent, depends on your organization, deployment strategy, etc!



Dependencies

Dependencies: external components for software to function correctly, but not directly controlled by development team. Modern software: lot of dependencies!

- **Examples**: Libraries (e.g., React, TensorFlow), Build tools (e.g., Maven, Gradle), External services (e.g., APIs, cloud platforms).
- **Dependency Hell**: when multiple dependencies conflict, when outdated libraries cause integration issues. Modern software numerous dependencies, increasing complexity of dependency management.
  - **Conflicting versions**: Two deps. require incompatible versions of same library.
  - **Circular dependencies**: Packages depend on each other in a loop.
  - **Deep dependency chains**: One small update can break many things down the chain.
  - **Breaking changes**: Updates in a dependency cause your software to fail.
- **Managing Dependencies** Strategies include: Embedding small or critical libraries directly into project, hosting medium-sized libraries in a private repository, using submodules or package managers for large, well-established libraries. Employing lockfiles and containers to ensure consistent runtime environments.

Deprecation

Process of orderly migration away from, eventual removal of obsolete systems. Planning, executing a transition strategy to minimize disruption.

- **Why Deprecate?** Code / Dependencies are **liability, not an asset**. Maintaining outdated systems incurs operational costs. Legacy systems hinder the evolution of newer systems due to compatibility requirements (quote Google). Allows orgs. to focus resources, maintain and improve current systems.
- **Challenges**: Emotional attachment to old systems. Difficulty in funding, executing deprecation efforts. High costs migrating to entirely new systems.
- **Types of Deprecation**: **Advisory Deprecation**: Inform users about deprecated features without enforcing immediate action. **Compulsory Depreciation**: Set deadlines for removal and ensure compliance through enforcement.
- **Deprecation During Design**: Design systems with deprecation in mind to simplify future decommissioning. E.g.: Plan for incremental replacement of components. Ensure clear documentation and migration paths for users (give alternative).

Code Review

Before deploying new code, critical to ensure quality, correctness. Unreviewed code: lead to bugs, poor design decisions, maintainability issues. Mitigate risks by involving others in process of inspecting, improving code.

- **Risk Mitigation**: Code reviews catch defects early, formal reviews identifying up to 60% of defects (comparatively - 30% for testing).
- **Maintainability**: Studies: up to 75% of code review comments address evolvability, maintainability, not just functionality.
- **Other Benefits**: Solicit feedback on code design, practices. Teaching, demonstrating coding methods. Raising awareness of product's state. Highlighting potential future problems.
- **Challenges**: *Speed vs. Thoroughness*: Review time increases exponentially with the size of the change, leading to diminishing returns on thoroughness. *Balancing Priorities*: if speed or depth of review more important.

Good Practices for Effective Code Reviews

- **Code Review Authors**: Keep reviews small and focused. Clear desc, annotate to guide reviewers. Choose appropriate reviewers, balance feedback quality w. team awareness. Open to feedback, implement suggested changes unless strong reason not. Reflect on feedback to improve future work.
- **Code Reviewers**: Push back on overly broad or poorly described changes. Focus on code, not person. Ask qns, point out problems, suggest solutions. Consider edge cases, failure scenarios, and future maintainability. Be responsive while balancing severity and importance of changes.

Software Quality Assurance (SQA)

Encompasses all activities aimed at ensuring product quality before reach customers. Unlike testing (finding bugs), SQA addresses broader aspects of quality, including usability, performance, and maintainability.

- **Key Principle**: Just as physical products quality checks, software products to meet high-quality standards before release.
- **SQA Scope**: test, docs, process adherence, customer UX validation.
- **Best Practices**: Simulate real-world customer usage to identify gaps between product, expectations. Ensure QA performed by someone other than developer to avoid biases. Focus on entire customer experience, not just crashes, functional bugs. (E.g. QA engineer tests edge cases (e.g., ordering -1 beers) and unexpected inputs (e.g., "ueicbksjdhd","where is toilet").)

Deployment

Deployment strategy impacts SWE practices. Method of deployment depends on factors e.g. update frequency, scalability, risk tolerance.

- **Examples of Deployment Methods**: Zip files on a server, Web apps w. continuous deployment ("DevOps"). App stores (mobile), Physical media (e.g. games), Pre-installed software on devices.
- **Considerations**: Speed, friction for releases/updates. Scalability to handle large user bases. Risks associated with deployment failures. Costs of maintaining infrastructure.
- **Deployment Strategy Examples**: *NASA*: Rigorous testing, version control, minimal runtime dependencies due to limited update opportunities. *Web App*: Allows instant updates, frequent releases, enabling rapid iteration.

Key Takeaways

- Learn best practices but adapt them to your organization's needs.
- There are no universal solutions; context matters.
- Teamwork, collaboration, and consistency outperform individual heroics.
- Think critically about organizational priorities when choosing practices.

Differences Between Junior and Senior Developers

- **Junior Devs**: Hide code until "ready." Provide inaccurate estimates, deliver inconsistently. Use long-lived branches, big merges. Large, unstructured commits w. unclear messages. View code review feedback as criticism.
- **Senior Devs**: Share code, ideas during dev. Provide accurate estimates, deliver on time. Use short-lived branches w. smaller merges. Make small, focused commits w. clear reasoning. View code review feedback as opportunity to improve.
- **Agility in Software Engineering**: "Solutions to common software engineering problems not always obvious. Agility - both technical and organizational - is essential to identify solutions that work for current challenges and remain resilient to future changes." - Asim Husain, VP Eng, Google.

8. Static Analysis and Type System

Limitations of Testing

- **Testing** widely used to identify bugs, but has significant limitations. Demonstrates presence of bugs but cannot guarantee their absence. Often time-consuming and resource-intensive. Edge cases and rare conditions may remain untested.
- **Historical Context of Software Failures**: significant financial losses and even endangerment of human lives. E.g. Mars Climate Orbiter (1998), Knight Capital Group (2012) trading glitch loss $440 million. Wannacry Ransomware Attack (2017). Boeing 737 MAX Crisis (2019).
- **Static analysis, type systems** can help play a crucial role in improving software reliability and reducing the incidence of costly bugs (Fewer runtime errors).

Static Analysis

- **Static analysis**: examination of code without executing it.
- **Primary goals**: identifying potential errors, adherence to coding standards, optimizing performance.
- **Why Static Analysis**: Early detection of bugs, reduce fix cost later in dev process. Improved code quality by enforcing best practices. Reduced debugging time through automated checks. Enhanced maintainability by ensuring consistent coding styles and fewer runtime errors.
- **Types of Static Analysis**:
  - **Linters and Style Checkers**: Focus on formatting and adherence to coding conventions without deeper semantic analysis.
  - **Data-Flow Analysis**: Tracks the flow of data through a program to determine possible values at different points in the execution.
  - **Control-Flow Analysis**: Analyze order of statement/function execution.
  - **Type Checking**: Verifies that variables and expressions are used consistently according to their declared types. (Focus on this)

The Problem with Unsafe Languages: E.g. Python, C, C++, and JavaScript, are inherently unsafe. These languages attempt to execute programs that may fail at runtime due to type mismatches or other errors.

- Python evaluates unsafe programs, throw runtime error when issues arise.
- C and C++ place the burden of writing safe programs entirely on the programmer, often leading to manual errors. (Manual proven uneffective)

Type System, Role in Static Analysis

- **A type system**: tractable syntactic method for proving absence of certain prog. behaviors by classifying phrases according to kinds of values they compute. (Benjamin Pierce, Types and Programming Languages)
- Type systems serve as critical component of static analysis, detect type-related errors before runtime. A type system acts as a filter.
- **Well-Typed Programs**: Adhere to type rules, allowed to execute. (e.g. $0 + 1$)
- **Ill-Typed Programs**: Prog w. Type mismatch reject before execution (e.g. $false\ 0$ - cannot apply boolean to integer).
- **Benefits of Type Systems**:
  - **Safety**: Prevents runtime errors caused by type mismatches.
  - **Early Error Detection**: Identifies errors during development than runtime.
  - **Improved Readability and Maintainability**: Clear type annotations make code easier to understand and modify.

λ-Calculus

- **Prog. language** invented in 1930s by Alonzo Church and Stephen Cole Kleene.
- "Core language" w. essential features to express all computation. (Lambda calculus is Turing complete). No redundancy, encode extra features as 'syntactic sugar.'
- Foundations of functional programming (e.g., Lisp, ML, Haskell).
- Often used as a core language to study language theories. (Type systems, Scope and binding, Higher-order functions, Denotational semantics, Program equivalence)

Overview: λ-Calculus as a Language

- **Syntax** (How to write program): Keyword "λ" for defining functions.
- **Semantics** (How to describe execution): *reduction* as calculation rules.
- **Main Points About λ-Calculus**: Succinct function expressions using λ.Bound variables can be renamed (α-equivalence). Reduction via substitution (β-reduction). Can be extended with: Types (e.g., simply-typed λ-calculus), and side-effects (not covered here).

Syntax

- λ-terms or λ-expressions: (Terms) $M, N ::= x \mid \lambda x.M \mid MN$
- **Lambda abstraction**: $(\lambda x.M)$: Anonymous functions.
- **Lambda application**: (MN).
- **Pure/Extended λ-Calculus**: Pure λ-calculus only has λ-terms. Extended has added extra operations and data types. (E.g. $\lambda x.(x+1)$, $\lambda z.(x+2y+z)$)
- **Conventions (Associativity)**
  - **Body of λ extends as far to the right as possible**: $\lambda x.MN$ means $\lambda x.(MN)$, not $(\lambda x.M)N$
  - **Function applications are left-associative**: MNP means $(MN)P$, not $M(NP)$
  - Examples: $(\lambda x.\lambda y.x - y)53 = ((\lambda x.\lambda y.x - y)5)3$ $(\lambda f.\lambda x.fx)(\lambda x.x+1)2 = ((\lambda f.\lambda x.fx)(\lambda x.x+1))2$

Higher-Order Functions

- Functions as **return values**: $\lambda x.\lambda y.x - y$
- Functions as **arguments**: $(\lambda f.\lambda x.fx)(\lambda x.x+1)2$
- **Example**: Function Composition: Given function f, return function $f \circ f$: $\lambda f.\lambda x.f(fx)$

$$(\lambda f.\lambda x.f(fx))(\lambda y.y+1)5 = (\lambda x.(\lambda y.y+1)((\lambda y.y+1)x))5$$
$$= (\lambda x.(\lambda y.y+1)(x+1))5 \quad \star\ \lambda \text{ extends as far right.}$$
$$= (\lambda x.x+1+1)5$$
$$= 5+1+1 = 7.$$

Curried Functions

- λ abstraction is a function of one parameter.
- $\lambda x.\lambda y.x - y \quad \leftrightarrow$ `int f(int x, int y) { return x - y; }`: Computationally equivalent (can be transformed into each other).
- **Curry**: Transform $(\lambda(x,y).x-y)$ to $(\lambda x.\lambda y.x-y)$. Uncurry: Reverse of Curry.

Free and Bound Variables

- E.g. $\lambda x.x + y$. x: Bound variable. y: Free variable.
- Bound variables renameable (α-equivalence): $\lambda x.(x+y) \Leftrightarrow \lambda z.(z+y)$
- Name of free variables matters: $\lambda x.(x+y) \neq \lambda x.(x+z)$
- **Occurrences**: $(\lambda x.x + y)(x+1) : x$ has both free and bound occurrence.

Processes in Lambda Calculus

1. **Alpha Conversion**: If are applying two lambda expressions w. same variable name inside, change one of them to a new variable name. E.g. $(\lambda x.xx)(\lambda x.x) \to (\lambda x.xx)(\lambda y.y)$
2. **Beta Reduction (Basically Substitution)**: Process of calling lambda expression with input, and getting output. E.g. $(\lambda x.xy)z \implies zy$. Substitution: $(\lambda x.M)N \to M[N/x]$. ('N replaces x in M'). Repeatedly apply the reduction rule to any sub-term.
3. **Eta Conversion/Eta Reduction**: Special case reduction. Converts between $\lambda x.(fx)$ and f whenever x does not appear free in f. Basically means $\lambda x.(fx) = f$ if f does not make use of x. E.g. $(\lambda x.(\lambda y.yy)x) = (\lambda y.yy)$

Formal Definitions

Terms:

$$M, N ::= x \mid \lambda x.M \mid MN$$

- **Set of free variables** in M: $fv(M)$

$$fv(x) := \{x\}, \quad fv(\lambda x.M) := fv(M) \backslash \{x\}, \quad fv(MN) := fv(M) \cup fv(N)$$

E.g.: $fv((\lambda x.x)x) = \{x\}, \quad fv((\lambda x.x+y)x) = \{x, y\}$

- **Substitution**: Only replace free variables.

$$M[N/x] : N \text{ replaces } x \text{ in } M$$

Substitution – Avoid Name Capture. Rename bound variables before substitution:

$$(\lambda x.x - y)[x/y] = (\lambda z.z - y)[x/y] = \lambda z.z - x$$

- **Alpha-equivalence**: $\lambda x.M = \lambda y.M[y/x]$, where y is fresh.. Two terms are α-equivalent iff one convertable to other purely by renaming bound variables.
- **Beta-reduction** (substitution): $(\lambda x.M)N \to M[N/x]$. ('N replaces x in M'). Repeatedly apply the reduction rule to any sub-term.

Normal Form

- **Reducible Expression** (β-redex): A term of the form $(\lambda x.M)N$.
- β-normal form: A term containing no β-redex.
- Stopping point: Cannot further apply β-reduction rules.
  E.g. $(\lambda f.\lambda x.f(fx))(\lambda y.y+1)2 \to 2+1+1$ (β-normal form)
  Can further reduce to 4 if reduction rules for $+$ are defined.
- E.g.: $\lambda x.\lambda y.x$ (normal), $(\lambda x.\lambda y.x)(\lambda z.z)$ and $\lambda x.(\lambda y.x)(\lambda z.z)$ (not normal)

Confluence (Church-Rosser Property)

- Terms can be evaluated in any order. Final result (if exists) is uniquely determined.
- 'When applying reduction rules to terms, the ordering in which the reductions are chosen does not make a difference to the eventual result.'
- **Confluence Theorem**: For all M, M_1, M_2, if $M \to^* M_1$ and $M \to^* M_2$, then $\exists M'$ such that $M_1 \to^* M'$ and $M_2 \to^* M'$.

Non-Terminating Reduction

- Some terms have no normal forms.
- E.g. $(\lambda x.xx)(\lambda x.xx) \to (\lambda x.xx)(\lambda x.xx) \to \ldots$
- Term may have both terminating and non terminating reduction sequences.

Reduction Strategies

- **Normal-order reduction**: Choose the left-most, outer-most redex first.

$$(\lambda u.\lambda v.v)((\lambda x.xx)(\lambda x.xx)) \to \lambda v.v$$

- Normal-order reduction will find a normal form if it exists.
- **Applicative-order reduction**: Choose the left-most, inner-most redex first.

$$(\lambda u.\lambda v.v)((\lambda x.xx)(\lambda x.xx)) \to (\lambda u.\lambda v.v)((\lambda x.xx)(\lambda x.xx)) \to \ldots$$

Reduction rules

Premise / Conclusion

$$\frac{}{(\lambda x.M)N \to M[N/x]}\ (\beta)$$

← *Function Application*

$$\frac{M \to M'}{M\ N \to M'N}$$

← *Left Reduction*

$$\frac{N \to N'}{M\ N \to M\ N'}$$

Repeatedly apply (β) to any sub-term

← *Right Reduction*

$$\frac{M \to M'}{\lambda x.M \to \lambda x.M'}$$

← *Function Body Reduction*

Soundness and Completeness

- **Soundness**: A sound type system never accepts a program that can 'go wrong.' Prevents false positives: The language is type-safe. Ensures execution never reaches a "meaningless" state.
- **Completeness**: A complete type system never rejects a program that cannot go wrong. Prevents false negatives.
- In practice, prioritize soundness, minimize false positives. (Impoosible for both)

9. Static Analysis and Type System

Static Analysis: A High-Level Perspective

Static analysis involves reasoning about code without executing it. Techniques:

- **Type Checkers**: Ensure variables used consistently w. declared types (Topic 8).
- **Linters and Style Checkers**: Enforce coding standards, formatting violations.
- **Data-Flow Analysis**: Tracks flow of data through program to identify issues.
- **Control-Flow Analysis**: Analyzes the possible execution paths of a program.

Static Analysis Tools in Practice: tools developed to detect style violations, potential bugs. E.g. **Java SpotBugs** (common bug patterns), **PMD** (code quality, e.g. overly complex methods, unused variables). Tools often face challenges such as false positives, false negatives, integration difficulties.

Soundness vs. Completeness in Static Analysis

- **Soundness**: If tool reports an issue, guaranteed to be true (no false positives).
- **Completeness**: If an issue exists, the tool will always report it (no false negatives).

Static analysis tools balance the two key properties. Achieving both is undecidable (no algorithm can definitively determine a "yes" or "no" answer for all possible inputs) in practice, leading to trade-offs between the two.

Challenges with Static Analysis

E.g. slow execution times, false positives (incorrectly flagged issues) and false negatives (missed bugs), limited applicability to complex features, unactionable warnings, lacking guidance for resolution, difficulty integrating into workflow:

- **Facebook's Infer Deployment**: Initial deployment, running Infer nightly 2,500 alarms ignored by devs due to workflow disruptions. Integrating Infer during code reviews (on diffs) increased fix rates to over 70%.
- **Google's FindBugs Deployment**: Bug dashboard outside workflows not used. Manually filed bug reports only 16% fixed. Lastly, integrated FindBugs in code reviews improved adoption but had false positives and inconsistent views.
- **Timeliness of reports** crucial; reports addressed when integrated into current tasks. Reporting new issues (on diffs) reduces noise, improve responsiveness.

Key Principles for Successful Static Analysis

To maximize the effectiveness of static analysis:

- Focus on developer happiness, ensure understandable, actionable outputs.
- Maintain low false positive rate (less than 10%) to build trust in the tool.
- Integrate seamlessly into developers' workflows (e.g., during code reviews).
- Provide timely feedback to ensure relevance to the current task.

Automated Testing **(Auto generate test input, and/or auto apply oracle)**

- Automated testing is preferred for complex systems (E.g. OS, compilers, DBMS), as manually written tests may miss bugs or may be incomplete.
- Example-based testing simpler but less comprehensive than property-based testing.
- **Black, Grey, White-box Approaches**: No standard definition
  - Black-box: no internal information (e.g., works on the binary only)
  - White-box: internal information (systematically analyze program)
  - Grey-box: some internal information (e.g., coverage)

Automated Testing: Property-Based Testing

Property-based testing involves specifying properties that the system should satisfy. The testing framework generates test inputs to check if these properties hold. (Test framework tries to find counterexample to break property / falsify it).

- **Example Property**: For a function `Math.abs(x)`, the property is $|x| \geq 0$. (Using jqwik, java property based testing framework).

```
@Property
void positiveAbs(@ForAll int val) {
    assertTrue(Math.abs(val) >= 0);
}
// arg0: -2147483648 is input that falsifies property.
```

- **Advantages**: Provides greater confidence than example-based unit tests. Useful for communicating specifications.
- **Challenges**: Requires creativity to define meaningful properties.
- Property based testing frameworks also allow constraining of input domain. Also provide ways to adapt existing generators or create new ones. Also provide shrinkers (test-input reduction tools).
- **In practice**: used primarily for testing components of complex systems. Especially in 'high-leverage' scenarios, where properties are easy to identify and test.
- **E.g.**: Google Translate, properties include round-trip translation similarity, idempotency (output deterministic same result for same input), word count etc.

Automated Testing: Differential Testing

Differential testing compares outputs from multiple implementations or versions of the same functionality, or a simple reference engine, to detect discrepancies.

- **Approach**: Common input, compare outputs of diff. versions, configurations, or implementations. If outputs differ, at least one system may be affected by bug.
- **Comparing Versions**: Check behavior against known-working base version ('golden version'), form of regression testing. Though change might be intended.
- **Comparing Configurations**: E.g. different optimization levels, different engines.
- **Comparing against Reference Engine**: E.g. mocked component.
- **Comparing Implementations**: For many important software systems, multiple (competing implementations exist). If sufficiently similar, they can be used for differential testing. (E.g. DB engines, XML processors, compilers.)
- **Challenges**: Small overlap in functionality between system can produce false alarms, may miss out common bugs (e.g. stem from common library used).

Automated Testing: Metamorphic Testing

Use source test case and its result to generate **follow-up test cases**, where differential testing might be inapplicable. Rely on **metamorphic relations** to infer expected results. Special case of property based testing. Black box approach.

- **Example Metamorphic Relation**: $\sin(\pi - x) = \sin(x)$. Test both $0.5, \pi - 0.5$.
- E.g. Test a sorting function whether relative order of sorted elements maintained when an element is removed from an input array. First result acts as oracle for second result.
- E.g. EMI (Equivalence Modulo Inputs) for Compiler testing, create programs that are equivalent only for a given input. Mutate unexecuted part of program running input I, and check if same output for I. Finds bugs as compiler optimizes across both executed and unexecuted parts.
- **Strengths**: Avoids need for multiple systems (unlike differential testing). Simple, effective if metamorphic relation is straightforward.
- **Weaknesses**: Finding bug-revealing metamorphic relations can be challenging.
- E.g. M. testing on Google Translate: Property: Exchange Nouns Preserves Sentence Structure.

Automated Testing: Fuzzing

Feed random/semi-random inputs to uncover crashes, unexpected behaviors.

- **Black-box Fuzzing**: Sends random inputs without knowledge of the program's internals. Simple, efficient test-case generation, but unlikely to uncover deep bugs especially if input format complex. (e.g. image parsers), as inputs probably invalid. (Partly addressed by grammar-based fuzzing).
- **Mutation-based Fuzzing**: mutate existing inputs over random inputs. Higher quality inputs with diverse input seed corpus, but requires seed inputs to mutate.
- **White-box Fuzzing**: White-box Fuzzing Techniques are often powered by constraint solvers and symbolic execution. Uses symbolic execution to systematically explore paths in the program.
- **Grey-box Fuzzing**: Instruments the program to collect code coverage and guide input generation. (E.g. American Fuzzy Lop (AFL), AFL++)
  - Balance the benefits of black-box fuzzing (efficient test case generation) and white-box fuzzing (high-quality inputs).
  - For each mutated input, check whether it resulted in a gain of code coverage (or code coverage pattern), if so, mutate further.
- **Fuzzing Test Oracles**: Crashes, hangs (pre-set thresholds), and dynamic analysis tools (detect issues e.g. race conditions, undefined behaviours).

Constraint Solvers

Powerful tools (building blocks) in other approaches: symbolic execution, white-box fuzzing, software verification etc. (E.g. Microsoft Z3 SMT solver).

- **Determine if formulas are satisfiable (SAT) or unsatisfiable (UNSAT)**. If a formula is satisfiable, solver provides a model— variables that satisfies formula.
- **SAT Solvers**: Solve propositional formulas with Boolean variables. For example, given the formula $(A \vee B) \wedge C$, a SAT solver might return $A = \text{true}, B = \text{false}, C = \text{true}$. (NP complete, but modern solvers efficient)
- **SMT Solvers**: Extend SAT solvers to handle more complex theories, such as linear integer arithmetic. (Satisfiability Modulo Theories) For example, given the formula $(a > 5) \wedge (a \neq 6)$, an SMT solver might return $a = 7$. (NP hard)
- **Challenges**: Solving SAT problems is NP-complete, SMT problems often NP-hard / undecidable for certain theories. Performance can degrade with complex path conditions or large formulas.

Symbolic Execution

Use an SMT solver to determine whether a path condition is feasible or not.

- **Key Idea**: Execute the program symbolically, maintaining a symbolic state (σ) that maps variables to symbolic expressions. Use a theorem solver to check the feasibility of path conditions and generate test cases for different execution paths.
- A path condition encodes all branch decisions on the symbolic variables taken so far. Path conditions can be feasible or infeasible.
- **Execution Tree**: Represents all possible execution paths of a program. Each branch corresponds to a decision based on symbolic variables. Path conditions encode the constraints on symbolic variables for each path. For example:

$$\alpha \wedge (\beta < 5) \wedge (\neg\alpha \wedge \gamma)$$

  Infeasible paths are pruned using an SMT solver. (E.g. α and $\neg\alpha$ above)
- E.g. KLEE Tool: popular symbolic execution tool for C programs built on the LLVM compiler framework. Running KLEE generates test cases that explore all feasible paths through the program.
- **Strengths**: Systematically generation of inputs to cover different execution paths.
- **Challenges**: Exponential Path Explosion, difficulty in Environment Modeling (library calls, external interacts), infinite Loops, Heap Modeling (Representing heap memory symbolically can be computationally expensive), Solver Performance.

Overall: Static analysis and automated testing are complementary approaches to improving software quality.